



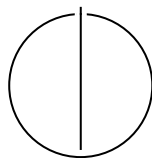
SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis, Master's Thesis, ... in Informatics

Thesis title

Author





SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

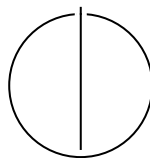
TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis, Master's Thesis, ... in Informatics

Thesis title

Titel der Abschlussarbeit

Author:	Author
Examiner:	Supervisor
Supervisor:	Advisor
Submission Date:	Submission date



I confirm that this bachelor's thesis, master's thesis, ... is my own work and I have documented all sources and material used.

Munich, Submission date

Author

Acknowledgments

Abstract

Contents

Acknowledgments	iii
Abstract	iv
1 Introduction	1
2 Background	3
2.1 Single Static Assignment	3
2.2 Liveness	3
2.3 Register Allocation	3
2.4 Register Allocation on SSA	4
2.5 TPDE	5
3 Related Work	6
3.1 Graphcoloring Register Allocation	6
3.2 Linear Scan Register Allocation	7
3.3 Trace Register Allocation	7
4 Approach	8
4.1 Fast Liveness	8
4.2 combining liveness sets and next-use distance	8
4.3 Spilling with next-use distance	8
4.4 Register Allocation with Repairing	8
4.5 VerificationIR (kind of useless)	8
5 Evaluation	9
5.1 Discussion	9
6 Conclusion	10
6.1 Future Work	10
Abbreviations	11
List of Figures	12

Contents

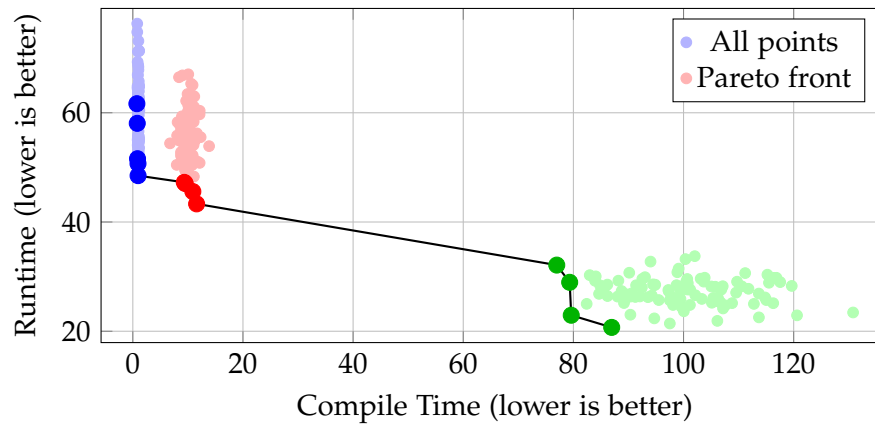
List of Tables	13
Bibliography	14

1 Introduction

- Compile speed is important for some applications like Database systems
- For short running programs, compile time is the dominant factor
- Gains from optimizations are not sufficient to amortize the additional compile time
- For long running programs, runtime is the dominant factor and more expensive optimizations are beneficial
- For most scenarios where a short compile time is wanted, the most interesting metric is the combined compile and runtime of a program [citation](#)
- TPDE [SKE25] is currently the fastest Compiler for LLVM-IR in unoptimized scenarios (-O0) targeting x84-64 and AArch64
- However, transitioning from TPDE to the next optimization level, LLVM [llvm] O1 pipeline is a large step in compile time
- compile time increases by approximately 10 times while delivering only around 5 times runtime improvement (as illustrated in Fig. ??)
- Therefore there is a significant subset of programs in JIT scenarios that are likely to benefit from some level of optimization, but currently lack a fast enough solution

Approach

- fast accurate liveness analysis [cite paper](#)
- utilize SSA form to separate spilling and register allocation phases
- Implement a fast spilling phase without requiring changes to the incoming IR.
- spilling analysis without requiring prior instruction selection phase.
- build a repairing register allocator to address architecture constraints.



Contribution

- Building a simplified spilling mechanism whichi guarantees correctness (even for incorrect liveness data)
- spilling wihtout modifications to IR
- Improving Pareto optimal compilation trade-offs.

Outline

- Background
- Liveness
- Spilling
- Register allocation
- Related work
- Evaluation
- Conclusion

2 Background

2.1 Single Static Assignment

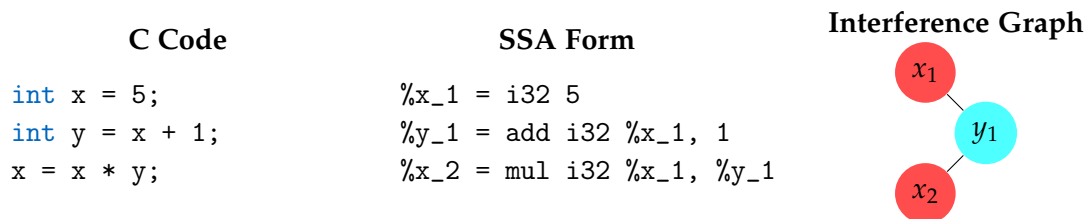


Figure 2.1: Example: C code and its LLVM-IR SSA representation

- Most modern compilers use an intermediate representation (IR)
- SSA is a popular form.
- In SSA each variable has 1 definition
- Simplifies transformations such as Dead Code Elimination
- Also has favourable properties for Register allocation
- LLVM-IR is in SSA.
- Phi nodes
-

2.2 Liveness

2.3 Register Allocation

- Register allocation is assigning physical registers to values from the IR.
- Traditionally modeled using Graph coloring as a mathematical model.

- Graph coloring is NP-hard in general [cite proof Regalloc np hard](#).
- In Graph coloring, directly connected nodes must have different colors.
- During register allocation, values that are live at the same time must have different registers.
- We can build an equivalent interference graph (IG) where nodes are values and edges exist between overlapping live ranges.
- If the graph can be colored with k colors, the equivalent program can be allocated with k registers without spilling (ignoring register constraints).
- Programs typically have more values than registers.
- Graph coloring register allocators typically try coloring the IG, then spill some live ranges (which reduces the number of edges)
- This process repeats until the graph can be colored with k colors.
- The example from Fig. 2.1 can be colored with 2 colors.
- The resulting program needs 2 registers. [chaitlin briggs allocator loop figure](#)
- For real architectures, we also need to consider register constraints.
- Some instructions require specific registers. For example the unsigned mul instruction on x86
- This can be modeled by adding additional edges and nodes to the IG.
- Due to the iterative nature and the need for an expensive IG data structure, traditional GC register allocators are not suited for JIT applications.

2.4 Register Allocation on SSA

- As independently discovered in [200x](#) SSA form, the resulting IG are chordal.
- This makes the graph coloring problem solvable in polynomial time.
- Notably it also allows for finding the maximal clique in polynomial time.
- A clique is a subset of nodes where every node is directly connected.

- Notably, any chordal graph can be colored with exactly the number of colors as the size of its maximal clique, which can be found in polynomial time.
- This allows for a new approach to register allocation.
- First reduce the number of all cliques to k (the number of available registers.)
- Second color the graph, which is now nearly trivial, [more detail how this works](#)
- Spilling and allocation are now 2 separate phases that each run once. It is no longer required to be iterative as in traditional graph coloring. [more detail](#)
- Register constraints make this more complicated, as the graph needs to remain chordal while representing the constraints.
- Depending on the specific constraints, this requires a lot of additional work [[hackthesis](#)].
-

2.5 TPDE

- Fast compiler framework.
- Can't modify IR.
- Encodegen

3 Related Work

all papers want instsel -> regalloc

3.1 Graphcoloring Register Allocation

As previously described, graph coloring of interference graphs is equivalent to register allocation [Chaitin1981]. In general this is an NP hard problem [Chaitin1981]. For graphs that are not colorable with k colors, the graph coloring algorithms additionally need to iterate between coloring, spilling and coalescing. Where each spill step requires an expensive recomputation of the interference graph [CD06]. This makes traditional graph coloring register allocation too slow for just in time compilation [CD06]. There are multiple improvements proposed to the original algorithm aiming to improve the code quality [study look cgo 2006](#).

The cost of rebuilding the interference graph after every spilling step can be avoided by approximating the resulting interference graph using additional metadata [CD06]. This sacrifices a small amount of code quality for a significant speedup, which makes it more suitable for just in time compilation. Still compared to other methods, the iterative nature of graph coloring is not desirable when prioritizing compilation speed. We leverage the SSA properties to do all spilling as a preprocessing step, and avoid building the interference graph entirely.

We will consider such SSA-based algorithms such as ours separately as they have

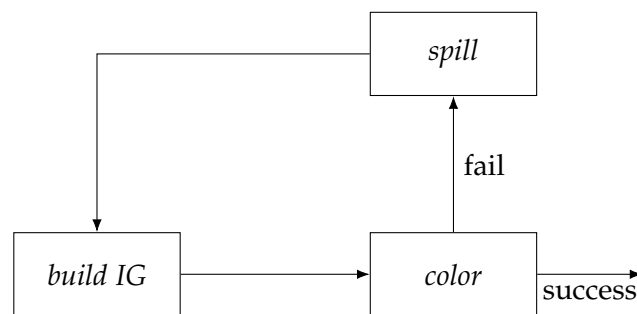


Figure 3.1: Simplified overview of the steps in a graph coloring allocator

different compilation speed properties, even though they are fundamentally based on graph coloring. [define graph RA](#) [define SSA](#) [define k](#)

3.2 Linear Scan Register Allocation

Linear Scan Allocators fundamentally of a single scan over the program, finding liveness and then mapping these live ranges to registers (or memory) [cite og linear scan](#). Although newer research present multiple improvements to the original algorithm [cite shiee cgo](#). Linear Scan Allocators are generally faster than graph coloring due to the single scan and no expensive interference graph construction. However, their code quality is generally worse than graph coloring [cite](#). While some of the limitations of the original algorithm have been addressed [[Wimmer2010](#)], Linear Scan Allocators fundamentally use a non-optimal approach with increasingly elaborate heuristics or post-processing passes to improve code quality [[Wimmer2010](#)].

Without a preprocessing step it is not possible for a linear scan to find the optimal spill placement around loops, which is particularly problematic as unnecessary spills and reloads in loops are more costly than outside of loops. Still it is popular choice for JIT Compilers. The original register allocator of TPDE [[SKE25](#)] could be considered a heavily modified linear scan allocator.

Our algorithm uses a preprocessing step using SSA properties to find better spill locations, especially around loops. As opposed to linear scan we use a greedy allocator, but with the additional theoretical guarantee that on a unconstrained architecture it will never cause an additional spill.

3.3 Trace Register Allocation

[citations](#) For JIT Compilers in particular, trace allocation can be used, which assigns registers on chosen segments incoming control flow, so called traces. This allows for fast algorithm and can use different allocation strategies based on the trace properties. However choosing good traces is non-trivial without runtime profiling information, and the resulting code quality is highly dependent on the trace selection.

As TPDE [[SKE25](#)] does not use have access to runtime profiling information, this approach is not suitable for our use case. Our approach allocates across control flow, without runtime information under the assumption that loops will be executed a multiple times.

4 Approach

4.1 Fast Liveness

A non iterative liveness analysis on strict SSA.

4.2 combining liveness sets and next-use distance

4.3 Spilling with next-use distance

- Irreducible control flow
- Spill on definition

4.4 Register Allocation with Repairing

- Multi- part challenges

4.5 VerificationIR (kind of useless)

5 Evaluation

5.1 Discussion

6 Conclusion

6.1 Future Work

- Register Aliasing
- Irreducible Control flow
- rest was ich nicht geschafft hab.
-

Abbreviations

List of Figures

2.1	Example: C code and its LLVM-IR SSA representation	3
3.1	Simplified overview of the steps in a graph coloring allocator	6

List of Tables

Bibliography

- [CD06] K. Cooper and A. Dasgupta. “Tailoring graph-coloring register allocation for runtime compilation.” In: *International Symposium on Code Generation and Optimization (CGO’06)*. 2006, 11 pp.–49. doi: 10.1109/CGO.2006.35.
- [SKE25] T. Schwarz, T. Kamm, and A. Engelke. *TPDE: A Fast Adaptable Compiler Back-End Framework*. 2025. arXiv: 2505.22610 [cs.PL].